



2.1. Predicados y Auxiliares

Ejercicio 1. Nombrar los siguientes predicados sobre enteros:

- a) $\text{pred } \text{????}(x : \mathbb{Z}) \{ (\exists c : \mathbb{Z}) (c > 0 \wedge c * c = x) \}$
- b) $\text{pred } \text{????}(x : \mathbb{Z}) \{ x > 1 \wedge (\forall n : \mathbb{Z}) (1 < n < x \rightarrow_L x \bmod n \neq 0) \}$

Ejercicio 2. Escriba los siguientes predicados sobre números enteros en lenguaje de especificación:

- a) $\text{pred } \text{divide}(x, y : \mathbb{Z})$ que sea verdadero si y sólo si x divide a y (es decir, el resto es 0).
- b) $\text{pred } \text{sonCoprimos}(x, y : \mathbb{Z})$ que sea verdadero si y sólo si x e y son coprimos.
- c) $\text{pred } \text{mayorPrimoQueDivide}(x : \mathbb{Z}, y : \mathbb{Z})$ que sea verdadero **si y sólo si** y es el mayor primo que divide a x .

Ejercicio 3. Nombre los siguientes predicados auxiliares sobre secuencias de enteros:

- a) $\text{pred } \text{????}(s : \text{seq}(\mathbb{Z})) \{ (\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L s[i] \geq 0) \}$
- b) $\text{pred } \text{????}(s : \text{seq}(\mathbb{Z})) \{ (\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L (\forall j : \mathbb{Z}) (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[i] \neq s[j])) \}$

Ejercicio 4. Escriba los siguientes predicados auxiliares sobre secuencias de enteros, aclarando los tipos de los parámetros que recibe:

- a) $\text{pred } \text{pertenece}(\dots)$ que determina si un elemento pertenece a una secuencia.
- b) $\text{pred } \text{esPrefijo}(\dots)$ que determina si una secuencia es prefijo de otra.
- c) $\text{pred } \text{estáOrdenada}(\dots)$ que determina si la secuencia está ordenada de menor a mayor.
- d) $\text{pred } \text{hayUnoParQueDivideAlResto}(\dots)$ que determina si hay un elemento primo en la secuencia que divide a todos los otros elementos de la secuencia.

Ejercicio 5. Escribir una auxiliar que

- a) Sume 10 a un entero.
- b) “Devuelva” el mayor elemento de una tupla de 3 enteros.
- c) “Devuelva” el dígito menos significativo de un entero.

Ejercicio 6. Sea s una secuencia de elementos de tipo $\mathbb{Z}$. Escribir una auxiliar utilizando sumatoria y productoria que:

- a) Cuento la cantidad de veces que aparece el elemento e de tipo $\mathbb{Z}$ en la secuencia s .
- b) Sume los elementos en las posiciones pares de la secuencia s .
- c) Sume los elementos mayores a 0 contenidos en la secuencia s .

2.2. Análisis de especificación

Ejercicio 7. Las siguientes especificaciones no son correctas. Indicar por qué y corregirlas para que describan correctamente el problema.

- a) `proc progresionGeometricaFactor2()` Indica si la secuencia l representa una progresión geométrica factor 2. Es decir, si cada elemento de la secuencia es el doble del elemento anterior.

```
proc progresionGeometricaFactor2(in l : seq(Z)) : bool {  
  requiere { Verdadero }  
  asegura { res = True  $\leftrightarrow$  ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |l| \rightarrow l[i] = 2 * l[i - 1]$ ) }  
}
```

- b) `proc mínimo()` Devuelve en res el menor elemento de l .

```
proc mínimo(in l : seq(Z)) : Z {  
  requiere { Verdadero }  
  asegura { ( $\forall y : \mathbb{Z}$ ) (( $y \in l \wedge y \neq res$ )  $\rightarrow y > res$ ) }  
}
```

Ejercicio 8. Para los siguientes problemas, dar todas las soluciones posibles a las entradas dadas:

- a) `proc indiceDelMaximo(in l : seq(R)) : Z {`

```
  requiere { |l| > 0 }  
  asegura {  $0 \leq res < |l| \wedge (\forall i : \mathbb{Z}) (0 \leq i < |l| \rightarrow l[i] \leq l[res])$  }  
}
```

- i) $l = \langle 1, 2, 3, 4 \rangle$
ii) $l = \langle 15.5, -18, 4.215, 15.5, -1 \rangle$
iii) $l = \langle 0, 0, 0, 0, 0, 0 \rangle$

- b) `proc indiceDelPrimerMaximo(in l : seq(R)) : Z {`

```
  requiere { |l| > 0 }  
  asegura {  $0 \leq res < |l| \wedge (\forall i : \mathbb{Z}) (0 \leq i < |l| \rightarrow (l[i] < l[res] \vee (l[i] = l[res] \wedge i \geq res)))$  }  
}
```

- i) $l = \langle 1, 2, 3, 4 \rangle$
ii) $l = \langle 15.5, -18, 4.215, 15.5, -1 \rangle$
iii) $l = \langle 0, 0, 0, 0, 0, 0 \rangle$

- c) ¿Para qué valores de entrada `indiceDelPrimerMaximo` y `indiceDelMaximo` tienen necesariamente la misma salida?

2.3. Relación de fuerza

Ejercicio 11. Sean x y res variables de tipo $\mathbb{R}$. Considerar los siguientes predicados:

P1: $\{x \leq 0\}$	Q1: $\{res \geq x^2\}$
P2: $\{x \leq 10\}$	Q2: $\{res \geq 0\}$
P3: $\{x \leq -10\}$	Q3: $\{res = x^2\}$

- a) Indicar la relación de fuerza entre P1, P2 y P3

- b) Indicar la relación de fuerza entre Q1, Q2 y Q3

c) Escribir dos programas que cumplan con la siguiente especificación:

```
proc hagoAlgo(in  $x : \mathbb{R}$ ) :  $\mathbb{R}$  {  
    requiere {  $x \leq 0$  }  
    asegura {  $res \geq x^2$  }  
}
```

d) Sea A un algoritmo que cumple con la especificación del ítem anterior. Decidir si necesariamente cumple las siguientes especificaciones:

- i) requiere { $x \leq -10$ }, asegura { $res \geq x^2$ }
- ii) requiere { $x \leq 10$ }, asegura { $res \geq x^2$ }
- iii) requiere { $x \leq 0$ }, asegura { $res \geq 0$ }
- iv) requiere { $x \leq 0$ }, asegura { $res = x^2$ }
- v) requiere { $x \leq -10$ }, asegura { $res \geq 0$ }
- vi) requiere { $x \leq 10$ }, asegura { $res = x^2$ }

e) ¿Qué conclusión pueden sacar? ¿Qué debe cumplirse con respecto a las precondiciones y postcondiciones para que sea seguro **reemplazar la especificación**?

2.4. Especificación de problemas

2.4.1. Problemas sencillos

Ejercicio 13. Especificar los siguientes problemas:

- a) Dado un entero, decidir si es par
- b) Dado un entero n y otro m , decidir si n es un múltiplo de m
- c) Dado un entero, listar todos sus divisores positivos (sin duplicados)
- d) Dado un entero positivo, obtener su descomposición en factores primos. Devolver una secuencia de tuplas (p, e) , donde p es un factor primo y e es su exponente, ordenada en forma creciente con respecto a p

Ejercicio 14. Dados dos enteros a y b , se necesita calcular su suma y retornarla en un entero c . ¿Cuáles de las siguientes especificaciones son correctas para este problema? Para las que no lo son, indicar por qué.

- a)

```
proc sumar(inout  $a, b, c : \mathbb{Z}$ ) {  
    requiere { Verdadero }  
    asegura {  $a + b = c$  }  
}
```
- b)

```
proc sumar(in  $a, b : \mathbb{Z}$ , inout  $c : \mathbb{Z}$ ) {  
    requiere { Verdadero }  
    asegura {  $c = a + b$  }  
}
```
- c)

```
proc sumar(inout  $a, b : \mathbb{Z}$ , inout  $c : \mathbb{Z}$ ) {  
    requiere {  $a = A_0 \wedge b = B_0$  }  
    asegura {  $a = A_0 \wedge b = B_0 \wedge c = a + b$  }  
}
```

2.4.2. Problemas sobre secuencias

Ejercicio 15. Especificar los siguientes problemas sobre secuencias:

- a) Dadas dos secuencias s y t , decidir si s está *incluida* en t , es decir, si todos los elementos de s aparecen en t en igual o mayor cantidad
- b) Dadas dos secuencias s y t , devolver su *intersección*, es decir, una secuencia con todos los elementos que aparecen en ambas. Si un mismo elemento tiene repetidos, la secuencia retornada debe contener la cantidad mínima de apariciones del elemento en s y en t .
- c) Dada una secuencia l con todos sus elementos distintos, devolver la secuencia de *partes*, es decir, la secuencia de todas las secuencias incluidas en l , cada una con sus elementos en el mismo orden en que aparecen en l .

Ejercicio 16. Dada una secuencia l , se desea sacar su primer elemento y devolverlo. Decidir cuáles de estas especificaciones son correctas. Para las que no lo son, indicar por qué y justificar con ejemplos.

- a)

```
proc tomarPrimero(inout l : seq⟨ℤ⟩) : ℤ {  
  requiere { |l| > 0 }  
  asegura { res = head(l) }  
}
```
- b)

```
proc tomarPrimero(inout l : seq⟨ℤ⟩) : ℤ {  
  requiere { |l| > 0 ∧ l = L0 }  
  asegura { res = head(L0) }  
}
```
- c)

```
proc tomarPrimero(inout l : seq⟨ℤ⟩) : ℤ {  
  requiere { |l| > 0 }  
  asegura { res = head(L0) ∧ |l| = |L0| - 1 }  
}
```
- d)

```
proc tomarPrimero(inout l : seq⟨ℤ⟩) : ℤ {  
  requiere { |l| > 0 ∧ l = L0 }  
  asegura { res = head(L0) ∧ l = tail(L0) }  
}
```

Ejercicio 18. Especificar los siguientes problemas de modificación de secuencias:

- a)

```
proc reemplazarParesPorSiguiente(inout l : seq⟨ℤ⟩)
```

 que reemplaza los elementos pares de la secuencia por el siguiente. Por ejemplo dada la secuencia $\langle 2, 7, 4 \rangle$ debería devolver $\langle 3, 7, 5 \rangle$
- b)

```
proc limpiarDuplicados(inout l : seq⟨char⟩) : seq⟨char⟩
```

, que elimina los elementos duplicados de l dejando sólo su primera aparición (en el orden original). Devuelve además una secuencia con todas las apariciones eliminadas (en cualquier orden)

2.4.3. Problemas sobre conjuntos y diccionarios

Ejercicio 19. Dado un conjunto de enteros, se espera que el siguiente predicado sea Verdadero cuando todos sus elementos son mayores a 0. Indicar por qué son incorrectas las siguientes especificaciones y proponer una valida.

- a)

```
pred sonTodosPositivos(c : conj⟨ℤ⟩) { (∀i : ℤ) (0 ≤ i < |c| → c[i] > 0) }
```
- b)

```
pred sonTodosPositivos(c : conj⟨ℤ⟩) { (∀e : ℤ) (e ∈ c ↔ e > 0) }
```
- c)

```
pred sonTodosPositivos(c : conj⟨ℤ⟩) { res = True ↔ (∀e : ℤ) (e ∈ c → e > 0) }
```

Ejercicio 20. Especificar los siguientes procedimientos:

- a) `proc sacarImpares(inout c : conj( $\mathbb{Z}$ ))` Que saca los elementos impares del conjunto c
- b) `proc devolverYEliminarMasChico(inout c : conj( $\mathbb{Z}$ )) :  $\mathbb{Z}$` Recibe un conjunto no vacio, elimina el menor elemento de esté y lo devuelve

Ejercicio 21. Dado un diccionario, se espera que el siguiente predicado sea Verdadero cuando no hay valores iguales para distintas claves. Indicar por qué son incorrectas las siguientes especificaciones y proponer una valida.

- a) `pred noHayValoresRepetidos(d : dicc( $K, V$ ))` { $(\forall c_1 : K) (c_1 \in d \wedge_L (\forall c_2 : K) (d[c_1] \neq d[c_2]))$ }
- b) `pred noHayValoresRepetidos(d : dicc( $K, V$ ))` { $(\forall c_1 : K) (c_1 \in d \rightarrow_L (\forall c_2 : K) (c_2 \in d \rightarrow_L c_1 \neq c_2))$ }
- c) `pred noHayValoresRepetidos(d : dicc( $K, V$ ))` { $(\forall c_1 : K) (c_1 \in d \rightarrow_L (\forall c_2 : K) (c_2 \in d \rightarrow_L d[c_1] \neq d[c_2]))$ }

Ejercicio 22. Dado el siguiente procedimiento:

`proc valorMasGrande(in d : dicc( $\mathbb{Z}, \mathbb{Z}$ )) :  $\mathbb{Z}$`

- a) Especificarlo, considerando que recibe un diccionario con tanto claves como valores números positivos, y da como resultado el valor más grande asociado a alguna clave del diccionario.
- b) ¿La solución propuesta permite que la entrada sea un diccionario vacío? Modifíquela para que, en dicho caso, devuelva -1.

Ejercicio 23. En nuestro lenguaje de especificación contamos con dos funciones útiles llamadas `setKey` y `delKey`, las cuales funcionan como detalla el apunte. Suponer que no existen dichas funciones y especificar los siguientes procedimientos:

- a) `proc eliminarClave(inout d : dicc( $K, V$ ), in c : K)` Que hace $d = \text{delKey}(D_0, c)$
- b) `proc agregarOModificarSiExiste(inout d : dicc( $K, V$ ), in c : K, in val : V)` Que hace $d = \text{setKey}(D_0, c, e)$

Ejercicio 24. Dado el siguiente procedimiento:

`proc modificarMayores(inout d : dicc( $\mathbb{Z}, \mathbb{Z}$ ), in umbral :  $\mathbb{Z}$ , in v :  $\mathbb{Z}$ )`

que cambia el valor de todas las claves de d que superen el umbral por el valor v, dejando las otras como estan, por ejemplo:

Umbral = 10, v = 100
 $D_0 = \{3 \rightarrow 2, 6 \rightarrow 12, 20 \rightarrow 0, 9 \rightarrow 1, 32 \rightarrow 5, 10 \rightarrow -2\}$
 $d = \{3 \rightarrow 2, 6 \rightarrow 12, 20 \rightarrow 100, 9 \rightarrow 1, 32 \rightarrow 100, 10 \rightarrow -2\}$

- a) Especificar el problema
- b) Para este problema es incorrecto utilizar `setKey`. Explique por qué y corrija su especificación en caso de haberlo usado.
- c) ¿Qué habría que agregar para que, a parte de modificar d, devuelva el conjunto de todas las claves cuyos valores fueron modificados? Es decir, $\{20, 32\}$ en el ejemplo dado.

2.5. Ejercicios de parciales anteriores

Ejercicio 25. Se desea especificar el problema `reemplazarNúmerosPerfectos`, que dada una secuencia de enteros devuelve la secuencia pero con los valores que se corresponden con números perfectos reemplazados por el índice donde se encuentran. Se llama números perfectos a aquellos naturales mayores a cero que son iguales a la suma de sus divisores positivos propios (divisores incluyendo al 1 y sin incluir al propio número). Por ejemplo, `reemplazarNúmerosPerfectos([0, 3, 9, 6, 4, 28, 7]) = [0, 3, 9, 3, 4, 5, 7]`, donde los únicos números reemplazados son el 6 y el 28 porque son los únicos números perfectos de la secuencia.

Ejercicio 26. Se desea especificar el problema `ordenarYBuscarMayor` que dada una secuencia s de enteros (que puede tener repetidos) ordena dicha secuencia en orden creciente de valor absoluto y devuelve el valor del máximo elemento. Por ejemplo,

- `ordenarYBuscarMayor([1, 4, 3, 5, 6, 2, 7]) = [1, 2, 3, 4, 5, 6, 7], 7`

- $\text{ordenarYBuscarMayor}([1, -2, 2, 5, 1, 4, -2, -10]) = [1, 1, -2, -2, 2, 4, 5, -10], 5$
- $\text{ordenarYBuscarMayor}([-10, -3, -7, -9]) = [-3, -7, -9, -10], -3$

Ejercicio 27. Se desea especificar el problema `procesarPrefijos` que dada una secuencia s de palabras y una palabra p , remueve todas las palabras de s que no tengan como prefijo a p y además retorna la longitud de la palabra más larga que tiene de prefijo a p . Por ejemplo, dados: $s = ["casa", "calamar", "banco", "recuperatorio", "aprobar", "cansado"]$ y $p = "ca"$ un posible valor para la secuencia s luego de aplicar `procesarPrefijos(s, p)` puede ser $["casa", "calamar", "cansado"]$ y el valor devuelto será 7.